30-Day PostgreSQL

Day 5 — Sorting & Limiting

Duration	~40 minutes
Level	Beginner
Week	1 — SQL Foundations
Goal	Sort, limit, and paginate query results for pipeline use cases

■ What You Will Learn

- ORDER BY ASC / DESC — control result order
- Multi-column sorting — tie-breaking across columns
- LIMIT — cap rows returned (sampling, top-N patterns)
- OFFSET — skip rows for pagination
- Pipeline patterns: watermark queries, batch processing

Session Agenda

Time	Segment	Key Points
0:00–3:00	Intro & recap	Review Day 4; why sort order matters in pipelines
3:00–12:00	ORDER BY	ASC/DESC, multi-column, tie-breaking
12:00–20:00	LIMIT	Top-N, sampling, LIMIT 1 watermark pattern
20:00–28:00	OFFSET	Pagination, batch processing caveat
28:00–35:00	Combining it all	WHERE + ORDER BY + LIMIT + OFFSET
35:00–40:00	Hands-on exercise + Q&A;	4 exercises, common mistakes, Day 6 preview

0:00 – 3:00

Segment 1 — Intro & Day 4 Recap

Welcome back. Yesterday we learned AND, OR, NOT, IN, and BETWEEN — the building blocks for combining filter conditions. Today we take control of **how results come back**: their order, how many rows, and which slice of the data we see.

Camera script:

"Before we sort anything, think about what would happen if you handed someone a 10-million-row table with no order. Chaos. Pipelines break, dashboards lie, incremental loads pick up from the wrong place. ORDER BY, LIMIT, and OFFSET are not nice-to-haves — they are essential tools for production data work."

3:00 – 12:00

Segment 2 — ORDER BY

Without ORDER BY, PostgreSQL returns rows in an unspecified order — it may look consistent in testing but will not be in production. Always sort explicitly when order matters.

Basic syntax:

```
SELECT product_name, price
FROM products
ORDER BY price ASC; -- low to high (ASC is the default)

SELECT product_name, price
FROM products
ORDER BY price DESC; -- high to low
```

Multi-column sort:

```
-- Sort by category A→Z, then price high→low within each category
SELECT product_name, category, price
FROM products
ORDER BY category ASC, price DESC;
```

The second column only activates when two rows tie on the first column.

Talking points:

- ASC is the default — you can omit it but being explicit is clearer
- Text columns sort alphabetically; numbers sort numerically; dates sort chronologically
- NULL values sort LAST by default in ASC order (NULLS LAST behaviour)
- Pipeline use: always sort by ingested_at DESC to inspect the newest rows first

---

12:00 – 20:00

Segment 3 — LIMIT

LIMIT caps the number of rows PostgreSQL returns. Critical for two reasons: (1) quick sampling of large tables, and (2) the watermark pattern that powers incremental loads.

```
-- Top 5 most expensive products
SELECT product_name, price
FROM products
ORDER BY price DESC
LIMIT 5;

-- Watermark pattern: most recent pipeline run
SELECT pipeline, started_at
FROM pipeline_runs
ORDER BY started_at DESC
LIMIT 1;
```

■ Pipeline Pattern: The Watermark Query

- Before each incremental load, run: ORDER BY loaded_at DESC LIMIT 1
- This gives you the 'high watermark' — the last timestamp loaded
- Next run: WHERE loaded_at > '' loads only new rows
- This avoids re-processing the entire table every time

Talking points:

- LIMIT without ORDER BY is unpredictable — always sort first
- LIMIT 1 is a pattern, not just a convenience — it is how pipelines find their position
- On large tables, LIMIT makes queries much faster by stopping early

---

20:00 – 28:00

Segment 4 — OFFSET & Pagination

OFFSET skips a number of rows before returning results. Combine with LIMIT to retrieve pages of data.

```
-- Page 1: rows 1–5
SELECT * FROM products ORDER BY price ASC LIMIT 5 OFFSET 0;

-- Page 2: rows 6–10
SELECT * FROM products ORDER BY price ASC LIMIT 5 OFFSET 5;

-- Page 3: rows 11–15
SELECT * FROM products ORDER BY price ASC LIMIT 5 OFFSET 10;

-- General formula: OFFSET = (page_number - 1) * page_size
```

■■ Performance Caveat

- OFFSET scans all skipped rows — it does NOT jump directly to position N
- On a 50-million-row table, OFFSET 40000000 scans 40 million rows first
- For production pipelines, prefer: WHERE id > last_seen_id (cursor-based pagination)
- OFFSET is fine for small datasets and reporting queries

28:00 – 35:00

Segment 5 — Combining Everything

The full clause order in a SELECT statement:

Clause	Purpose	Example
SELECT	Choose columns	SELECT name, price
FROM	Choose table	FROM products
WHERE	Filter rows	WHERE category = 'electronics'
ORDER BY	Sort results	ORDER BY price DESC
LIMIT	Cap row count	LIMIT 10
OFFSET	Skip rows	OFFSET 20

Full example — pipeline inspection query:

```
-- Most recent 10 successful orders_load runs
SELECT run_id, pipeline, rows_loaded, started_at
FROM pipeline_runs
WHERE pipeline = 'orders_load'
AND status = 'success'
ORDER BY started_at DESC
LIMIT 10;
```

35:00 – 40:00

Segment 6 — Hands-On Exercise

Using this inline products dataset:

```
SELECT * FROM (VALUES
(1, 'Widget A', 'electronics', 29.99),
(2, 'Widget B', 'clothing', 14.50),
(3, 'Widget C', 'electronics', 89.99),
(4, 'Widget D', 'food', 4.99),
(5, 'Widget E', 'clothing', 34.00),
(6, 'Widget F', 'electronics', 12.00),
(7, 'Widget G', 'food', 22.75),
(8, 'Widget H', 'clothing', 99.99)
) AS products(product_id, product_name, category, price);
```

Tasks:

#	Task
1	All products sorted by price DESC
2	Top 3 most expensive products
3	Sort by category ASC, then price DESC within each category
4	2nd and 3rd cheapest products (OFFSET 1 LIMIT 2)

Expected outputs:

Task 1 — 8 rows, Widget H ($99.99) first, Widget D ($4.99) last

Task 2 — 3 rows: Widget H $99.99 | Widget C $89.99 | Widget E $34.00

Task 3 — 8 rows grouped by category, sorted by price within each

Task 4 — 2 rows: Widget F $12.00 | Widget B $14.50

Common mistakes to watch for:

Mistake	What Happens	Fix
LIMIT without ORDER BY	Unpredictable rows returned	Always sort before limiting
Wrong OFFSET for page N	Skips wrong number of rows	OFFSET = (N-1) × page_size
ORDER BY after LIMIT	Syntax error	ORDER BY always comes before LIMIT
OFFSET on huge tables	Slow — scans all skipped rows	Use WHERE id > last_id instead

Session Recap

✓ ORDER BY sorts results — ASC (low→high) or DESC (high→low)

✓ Multi-column ORDER BY: second column breaks ties from the first

✓ LIMIT caps the row count — always pair with ORDER BY

✓ OFFSET skips rows for pagination — use with caution on large tables

✓ LIMIT 1 + ORDER BY DESC = watermark query — the backbone of incremental loads

✓ Clause order: SELECT → FROM → WHERE → ORDER BY → LIMIT → OFFSET

■ Day 6 Preview — String Functions

• LOWER(), UPPER(), TRIM() — normalise text from messy sources
• CONCAT() and || — build strings from column values
• LIKE and ILIKE — pattern matching on text columns
• Real use case: cleaning a customer name column from a CSV import

Camera script (outro): "Today you learned how to control your results — order them, cap them, and page through them. The LIMIT 1 watermark pattern alone will save you hours of re-processing data. Tomorrow we clean messy strings. See you on Day 6."

Appendix — Quick Reference Card

Clause / Pattern	Syntax	Pipeline Use Case
Sort ascending	ORDER BY col ASC	Oldest events first
Sort descending	ORDER BY col DESC	Most recent records first
Multi-column sort	ORDER BY a ASC, b DESC	Group by pipeline, then newest run
Top-N rows	ORDER BY col DESC LIMIT N	Top 10 customers by revenue
Watermark query	ORDER BY ts DESC LIMIT 1	Find last loaded timestamp
Page K (size N)	LIMIT N OFFSET (K-1)*N	Batch processing chunks
Skip nulls + sort	WHERE col IS NOT NULL ORDER..	Clean sort on nullable column